CS2109S Introduction to AI. and ML

AY24/25 Sem 2. github.com/gerteck

- **Search Algorithms** (Uninformed Search (BFS/DFS), Local Search (Hill Climbing), Informed Search (A*), Adversarial Search (Minimax))
- **"Classical" ML** (Decision Trees, Linear/Logistic Regression, Kernels and Support Vector Machines, "Classical" Unsupervised Learning)
- **"Modern" ML** (Neural Networks, Deep Learning, Sequential Data)

1. Introduction to AI

Intelligent Agent and Environments

An agent perceives environment through sensors, act through actuators.
Rational: choose actions to maximize expected performance.

- **Percept**: content agent's sensors are perceiving. **Percept Sequence** complete history of things agent has perceived.
- **PEAS**: Performance Measure (definition of success), Environment (world agent operates in), Actuators (actions agent can perform), Sensors (Inputs agent gets from environment)
  (E.g. Chatbot: Sensor as text input/chat history, Actuators as output, Environment as chat interface, plugins, Perf. measure as correctness safety.)
- **Exploration vs. Exploitation**: Exploration: learn more about world. Exploitation: Maximize gain based on current knowledge.

Properties of Task Environments

Observable: Fully vs. Partially observable.
Deterministic vs. Stochastic: Next state determined by current state and action. If environment also dependent on actions of other agents, also *strategic*, unless other agents are predictable.
Episodic vs. Sequential: Single-step vs. multi-step tasks. (Next episode does/does not depend on actions taken in previous episode.)
Static vs. Dynamic: Environment changes while deliberating. - semi-dynamic (env no change, only change agent performance score)
Discrete vs. Continuous: Finite (distinct, clearly defined) vs. infinite (states/percepts)/actions.
Single-agent vs. Multi-agent: Operating alone or with others.

Common Agent Structures

Simple Reflex Agent: Condition-action rules (e.g., "If up empty: up"). Based on current percept, ignore percept history.
Goal-Based Agent: Acts to achieve goals. Tracks state of world, goals.
Utility-Based Agent: Maximizes utility (e.g., rewards). Use utility function (perf. measure) to assign scores.
Learning Agent: Adapts and improves over time, using performance element, critic and problem generator.

2. Problem Solving by Searching

Search Problem

Search problem defined where goal is to find state/(path to state) from a set of possible states.
State Space. Set of possible states environment can be.
Representation Invariant: Abstract states hv. mapped concrete states.
Initial State where agent starts, **Goal State(s)**: can be > 1, use goal test.
Actions. Given state s, finite set of actions that can be executed.
Transition Model. Describes what each action does. Returns state resulting from action a in state s.
Action Cost Function: Numeric cost of applying action to reach next state.
Search Algorithm: Takes search problem as input, returns a solution / failure. Solution could be sequence of actions, or final state. Optimal solution is one with least cost.

Search Algorithms

Search algorithms explore the state space to find solution. Key operations:
Frontier: Set of nodes to explore next. *Explored Set*: Nodes already visited.
Node: Contains state, parent node, action, path cost, and depth.

Performance/Evaluation
Completeness: Finds solution if exists / report failure for none.
Optimality (Cost) : If outputs solution, it is lowest path cost of all solutions. Algorithm can be both optimal and incomplete.
Time, Space complexity: How long, how much memory needed.

Uninformed Search Algorithms

No information to guide search, aka blind search, no clue how good state is.

Breadth-First Search (BFS)

- **Frontier**: Queue (FIFO). Explore all nodes at curr. depth, before deeper.
- **Complete**: if branching factor b finite. **Optimal**: if step cost uniform.
- **Complexity**: Time: $O(b^d)$, d is depth of shallowest sln. Space: $O(b^d)$.
- *Optimize*: track visited set, early goal test (check node soln once generated).

Uniform-Cost Search (UCS) (Dijkstra's SSSP 'equiv')

- **Frontier**: Priority queue by min total path cost. Expand least-cost first.
- No-visited-mem ('tree-search', as if tree). Visited-mem ('graph-search').
- Even if goal state in frontier, sorted, first g-state selected must be optimal.
- **Complete**: if step costs are positive. **Optimal**: if step costs are positive.
- **Complexity**: Time: $O(b^{C^*/\epsilon})$, C^* is the optimal cost, ϵ is minimum step cost. Space: $O(b^{C^*/\epsilon})$. (w.r.t 'tier' of optimal solution)

Depth-First Search (DFS)

- **Frontier**: Stack (LIFO). Explores as deep as possible before backtrack.
- **Incomplete**: if search tree has infinite depth or loops.
  Non-optimal: No, may find suboptimal solutions.
- **Complexity**: Time: $O(b^m)$, where m is max depth. Space: $O(bm)$.

Depth-Limited Search (DLS)

- Limit search depth to l. Backtrack when limit hit. Good if opt. d known.
- **Incomplete**: if sln exists beyond depth l. **Non-optimal**: if used w. DFS.
- **Complexity**: Time: $O(b^l)$. Space: $O(bl)$.

Iterative Deepening Search (IDS)

- Repeatedly applies DLS with increasing depth limits. Try all possible depth limits from 0 till sln found or failure value returned from DLS. Combines benefits of BFS and DFS.
- **Complete**: If branching factor b finite. **Optimal**: if step costs uniform.
- **Complexity**: Time : $O(b^d)$. Space: $O(bd)$.

Informed Search Algorithms

Guide search with extra info, aka heuristic function on how close to goal.
Heuristic $h(n)$ estimates cost to reach goal from node n. $h^*(n)$: true cost.

- **Admissible**: Never overestimates cost ($h(n) \leq h^*(n)$). (e.g. $h(n) = 0$)
- **Consistent**: if for every node n, every successor n' of n generated by any action a: $h(n) \leq c(n, a, n') + h(n')$ and $h(G) = 0$. Δ Inequality.
- *Relaxed Problem*: Problem w. fewer restrictions. Cost of optimal solution to relaxed problem is admissible heuristic for original problem. (e.g. straight line distance for cities distances).
- *Dominance*: If $h_1(n) \geq h_2(n)$ for all n, h_1 dominates h_2. Dominant heuristics better for search if admissible. (e.g. straight line distance, vs trivial heuristic $h = 0$.)

Best-First Search

- **Frontier**: Priority queue ordered by evaluation function $f(n)$. Expands most promising node based on just $f(n)$ (no past cost).
- Not complete, not cost-optimal.

A* Search

- **Frontier**: Priority queue. (Best-First Search considering path cost).
- Use evaluation $f(n) = g(n) + h(n)$. $g(n)$ is cost to reach n from start. $h(n)$ is estimated cost to reach goal from n.
- **Complete**: if $h(n)$ is admissible, state space finite, step costs positive. **Optimal** if $h(n)$ is admissible.
- **Complexity**: Time: Exponential but improved by good heuristics. Space: Exponential. (Both $O(b^d)$ for poor heuristic.)

If $h(n)$ admissible, A* search (w/o visited memory) is optimal. If $h(n)$ consistent, A* search (w. visited memory) is optimal.

- A consistent heurisitic $\implies$ admissible (T2.QnC.1). Converse not true.
- (Trivial $h = 0$ consistent, admissible)
- Triangle inequality: Sum of lengths of any two sides $\geq$ length of 3^{rd} side.

3. Local Search

For (NP-hard) problems, traverse state space in systematic manner not option, is intractable (unsolvable in time).

Local Search: Explore state space considering only locally-reachable states ('search space'). Start random position, iterative move to neighboring states via perturbation or construction. Solution is final state, not path taken.
(E.g. N-Queens Problem, minimize function value)

- Typically incomplete and suboptimal.
- *Any-time property*: Longer runtime yields better solutions. Suitable for obtaining "good enough" solutions for difficult problems.
- **Types**: *Perturbative* Search (search space is complete solutions, modify components of current solution). *Constructive* Search (partial candidate solutions, extend by adding components).

Problem Formulation in Local Search:
States: Represent configurations/candidate sln within search space, but may not directly map to actual state. *Goal Test* (Optional) checks if current state is desired solution. *Successor Function*: Generates neighboring states by applying small modifications.
Evaluation/Objective Function: assess quality of state, max/minimize based on problem. (E.g. N-Queen Maximize "safe" queens, min. y value.)

Hill-Climbing Algorithm

Find local maxima by travelling to neighbouring states with highest value, terminate when no neighbour has higher value. One objective function is to negate heuristic function.

```
current_state = initial_state
while True:
    best_successor = highest-valued successor of curr_state
    if eval(best_successor) <= eval(current_state):
        return current_state
    current_state = best_successor
```

Greedy local search: gets stuck at local maxima, ridges, plateaus, shoulders (same values flat area). Some solutions include limited sideways move, stochastic (probability based on steepnes of move), first-choice, random-restart. Simulated Annealing - allow occasional 'bad moves' to escape local optima.

4. Adversarial Search and Games

'Classicial' Adversarial: Agents/Player compete against each other. Look at deterministic, two player, turn-taking, perfect information, zero-sum games. Perfect information: fully observable. Zero sum: no "win-win".

- *States*: Configurations of game, initial state, terminal state (win/lose/draw).
- *Actions*: Possible moves by a player.
- *Transition Model*: Defines how actions change the state.
- *Utility Function*: Assigns a value to terminal states from perspective of the agent. (E.g. Win: $+1$, Draw: 0, Loss: -1)

Minimax Algorithm

Determine optimal move for Player A (MAX) assuming Player B (MIN) plays optimally. Work out minimax value of each state in tree.

- **Complexity**: Time: Exponential w.r.t. max depth ($O(b^m)$). Space: Poly.
- **Complete**: If tree is finite. **Optimal** if against optimal opponent.

It is a recursive algo, backs up minimax values as recursion unwinds. Thus, similar to DFS, time c. $O(b^m)$, space c. $O(bm)$.
Note: minimax not 'optimal' against non-optimal MIN player, as non-optimal p could choose higher value in path pruned as it had lower min value. But guarantees it never has lower utility than that of optimal opponent.

Alpha-Beta Pruning (Minimax)

Reduces number of nodes (computational overhead) evaluated in game tree without affecting final decision. Minimax explores full game tree, not needed.

- α: Best value (highest) MAX can guarantee.
- β: Best value (lowest) MIN can guarantee.
- **Algorithm**:

```
def max_value(state, \alpha, \beta):
    if is_terminal(state): return utility(state)
    v = $-\inf$
    for next_state in expand(state):
        v = max(v, min_value(next_state, \alpha, \beta))
        \alpha = max(\alpha, v)
        if v >= \beta: return v
    return v

def min_value(state, \alpha, \beta):
    if is_terminal(state): return utility(state)
    v = $\inf$
    for next_state in expand(state):
        v = min(v, max_value(next_state, \alpha, \beta))
        \beta = min(\beta, v)
        if v <= \alpha: return v
    return v
```

Good move ordering improves effectivness of pruning.

Evaluation Function to Cutoff (Minimax)

Fully exploring game tree with Minimax may be computationally infeasible.

- Apply **cutoff strategy** to halt the search at predefined maximum depth.
- Use **heuristic function** to estimate the utility of mid-game (non-terminal) states. Heuristic must be some value between win and loss, and computationally efficient. Admissibility, consistency properties do not apply. Can use domain-specific features or some labeled dataset to train heuristics.

5. Machine Learning

Computers with ability to learn without explicit programming, through algorithms that learn from, and make predictions based on data. Goal is for performance to improve over time by identifying patterns in the data.

Problem Types

- **Classification**: Output category value in finite set. (Predict discrete label)
- **Regression**: Output is number. (Predict continuous num. value)

Types of Machine Learning (Feedback difference)

1. **Supervised Learning**: Learns from labeled data (input-output pairs) to map inputs to outputs. (E.g. classify pic banana or apple)
2. **Unsupervised Learning**: Learns from unlabeled data to find patterns or structure. (E.g. clustering, group based on features).
3. **Semi-Supervised Learning**: Use labeled, unlabeled data for learning.
4. **Reinforcement Learning**: Learns to make decisions by interacting with an environment, (rewards and penalties).

Supervised Learning

- **Training Phase**: Learn to map inputs to outputs by minimizing difference btw. predictions and ground truth using a training set.
  Result of training called a *model / hypothesis*.
- **Testing/Evaluation Phase**: Measures the model's performance on unseen data (test set) to evaluate generalization.
- Supervised learning: dataset represented as set of pairs $(x^{(i)}, y^{(i)})$:
  - $x^{(i)} \in \mathbb{R}^d$: Input vector of i-th data point with d features.
  - $y^{(i)}$: Label (target) for the i-th data point. ($y^{(i)} \in \mathbb{R}$ or $\in \{1, \ldots, C\}$)
  - Dataset D with n examples: $D = \{(x^{(1)}, y^{(1)}) \ldots, (x^{(n)}, y^{(n)})\}$.

Assume unknown true relationship $y = f^*(x) + \epsilon$, ϵ noise error term. Find hypothesis $h(x)$ that approximates true function f^*.

- **Hypothesis class** $\mathcal{H}$: set of all possible learnable models or functions. Each $h \in \mathcal{H}$ is a hypothesis or model. (Classes - e.g decision tree, linear model, neural networks).
- **Learning Algorithms** A: takes training set D_{train}, find hypothesis $h \in \mathcal{H}$. (E.g. Decision tree learning, gradient descent algorithms)
- **Performance Measure**: Test hypothesis on new set (test set). Metrics:
  - (Regression) Mean Squared Error (MSE): $= \frac{1}{N} \sum_{i=1}^{N} (\hat{y}^{(i)} - y^{(i)})^2$.
  - (Regression) Mean Absolute Error (MAE): $= \frac{1}{N} \sum_{i=1}^{N} |\hat{y}^{(i)} - y^{(i)}|$.
  - (Classification) Accuracy: Accuracy $= \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}(\hat{y}^{(i)} = y^{(i)})$.
  - (Classification) Confusion Matrix: e.g. Precision ($\frac{TP}{TP+FP}$ - maxm if FP costly), Recall ($\frac{TP}{TP+FN}$ - maxm if FN costly), F1 Score: $\frac{2}{\frac{1}{P}+\frac{1}{R}}$

6. Decision Trees

Function that takes as input a vector of attribute values and returns a decision (single output value). Perform sequence of tests from root node to leaf.

- **Expressiveness**: Can express any (propositional) fn of input attributes.
- **Size of Hypothesis Class**: Distinct decision trees, n bool. attributes: 2^{2^n}
- **Decision Tree Learning**: Greedy, top-down, recursive algorithm. Use information gain, choose best attribute to divide training set. For each value of chosen attr. use DTL again on corresponding subset of examples.
- **Entropy**: $I(P(v_1), P(v_2), ..., P(v_k)) = -\sum_{i=1}^{k} P(v_i) \log_2 P(v_i)$
- **Information Gain**: Initial Entropy $-$ avg. entropy after divide 'remainder'. Entropy before dividing: $I(\frac{p}{p+n}, \frac{n}{p+n})$, $p+n$ examples in subset E_i.

Information Gain

Given *attribute* A with v distinct values, the training set is split into subsets $E_1, ..., E_v$, each with p_i positive and n_i negative examples.

- remainder$(A) = \sum_{i=1}^{v} \frac{p_i+n_i}{p+n} I\left(\frac{p_i}{p_i+n_i}, \frac{n_i}{p_i+n_i}\right)$
- Info Gain: $IG(A) = I(p_{\text{pos}}, p_{\text{neg}}) - \text{remainder}(A)$

Choose attribute with max information gain.
Generalisation/Overfitting: **Prune**, reduce size of decision tree.
- **min-sample leaf** (min threshold of sample count in leaf node)
- **max depth** (longest path from root to leaf) pruning.

Smaller DT may higher accuracy as sample due to noise ignored.
Data Preprocessing: Missing values (assign most common value, drop row/attribute etc.) Continuous values - partition to discrete set as attribute.